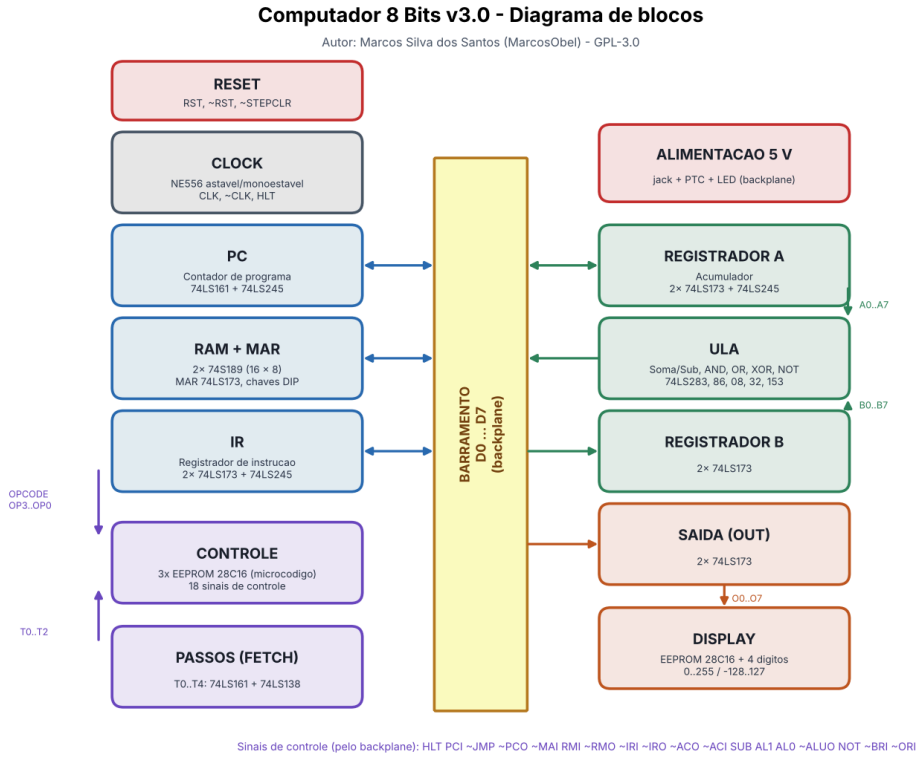
Computador 8 Bits

Manual completo — arquitetura, fabricação, montagem, gravação, uso e testes



Marcos Silva dos Santos (MarcosObel)

github.com/aragonxpd154/8-bits-computer · GPL-3.0 · v3.0 · 2026

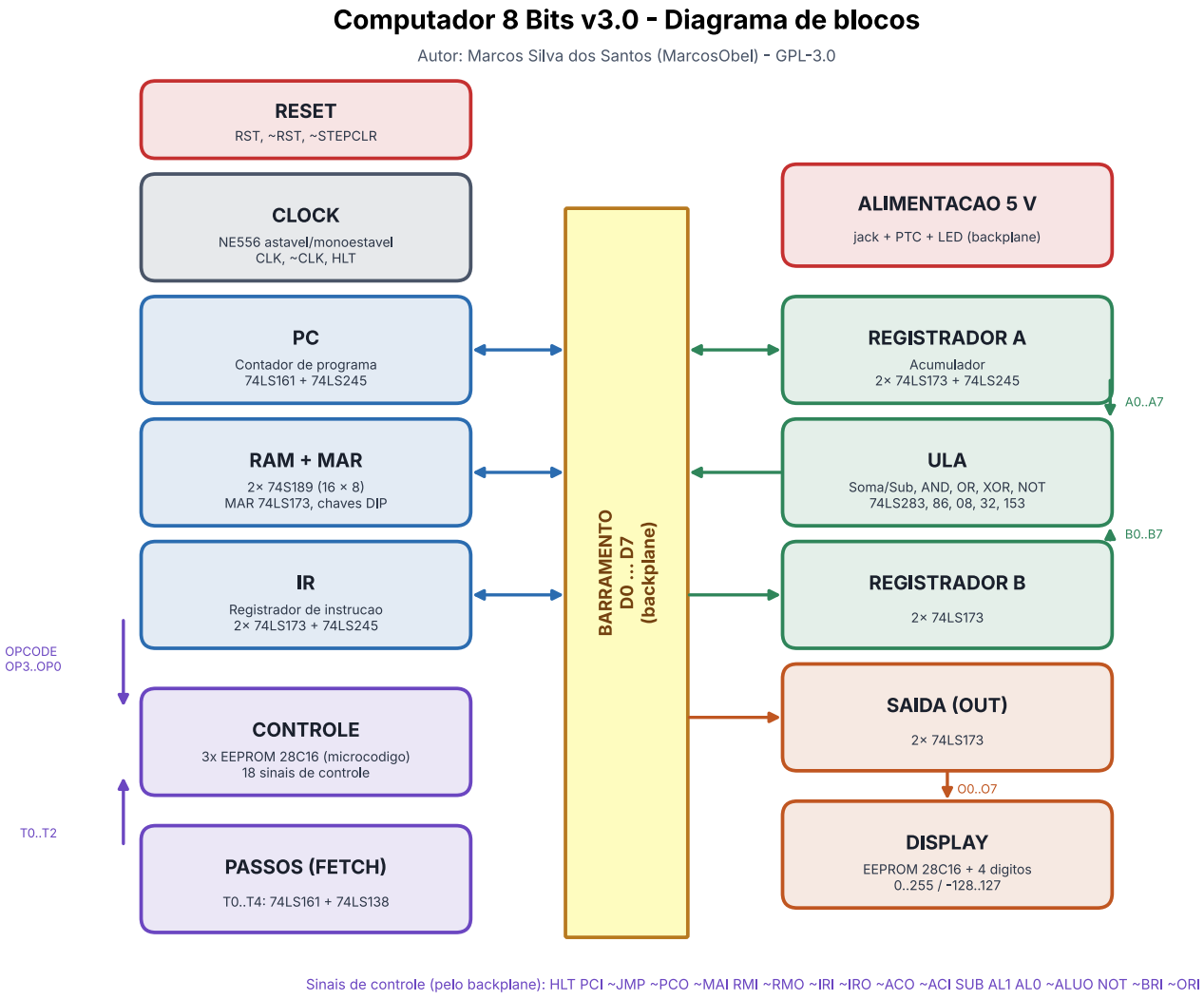
1. Arquitetura do Computador 8 Bits
   1.1 Os módulos
   1.2 Como uma instrução é executada
   1.3 Conjunto de instruções
   1.4 Sinais de controle
   1.5 Barramento e ligações diretas
2. Revisão do projeto original e correções da versão 3.0
   2.1 Erros que impediam o computador de funcionar
   2.2 Melhorias de confiabilidade
   2.3 Verificações automáticas feitas
   2.4 Checklist de revisão (atualiza o do README original)
3. Fabricação das placas (JLCPCB / PCBWay)
   3.1 O que a placa mostra (pensado para estudo)
   3.2 Arquivos (em `hardware/fabricacao/<placa>/`)
   3.3 Pedido no JLCPCB (o PCBWay é praticamente igual)
   3.4 Quando as placas chegarem
   3.5 Apêndice — fazer em casa (opcional)
4. Montagem das placas — componente por componente
   4.1 Regras gerais (valem para todas as placas)
   4.2 Teste de alimentação (fazer em TODAS as placas)
   4.3 Montagem e teste de cada placa
5. Interligação: backplane, cabos e montagem na base
   5.1 Ideia geral
   5.2 Arranjo físico sugerido
   5.3 Alimentação
   5.4 Tabela de cabos
   5.5 Ordem de ligação recomendada
6. Gravação das EEPROMs (no Zorin OS / Ubuntu)
   6.1 Gerar (ou regerar) os binários
   6.2 Gravar com TL866II Plus / XGecu T48 usando `minipro`
   6.3 Sem gravador?
   6.4 Conferir sem o computador montado
7. Programação e uso
   7.1 Painel de controle (onde fica cada coisa)
   7.2 Escrever um programa
   7.3 Gravar o programa na RAM (pelas chaves)
   7.4 Executar
   7.5 Programas de exemplo (`firmware/programas/`)
   7.6 Conjunto de instruções (resumo)
8. Testes e diagnóstico
   8.1 Ferramentas
   8.2 Subida do sistema (bring-up) em etapas
   8.3 Problemas comuns
   8.4 Medidas de referência

Lista de materiais completa - Computador 8 Bits v3.0
   Totais (todas as 13 placas)

Por placa

1. Arquitetura do Computador 8 Bits

> Projeto de **Marcos Silva dos Santos** (MarcosObel) — GPL-3.0.



> 🧊 Cada bloco tem uma página própria, com diagrama de funcionamento e a placa em 3D: guia das placas.

Um computador de 8 bits construído só com lógica TTL da série 74LS, memórias EEPROM 28C16 e temporizadores 555 — **sem microcontrolador e sem CPU pronta**. A arquitetura segue a família "SAP-1" (*Simple As Possible*), com algumas extensões (operações lógicas na ULA, saltos condicionais preparados, display decimal com sinal).

Característica	Valor
Largura de dados / barramento	8 bits (D0…D7)

Característica	Valor
Endereçamento	4 bits → 16 bytes de RAM (74S189 ×2)
Registradores	A (acumulador), B, IR, OUT, MAR, PC
Instruções	16 opcodes (14 em uso + 2 reservados)
Microinstruções	até 5 passos por instrução (T0…T4), 18 sinais de controle
Clock	0,7 Hz a ~140 Hz (automático) ou passo a passo (manual)
Saída	display de 4 dígitos, 0…255 ou −128…127
Alimentação	5 V, ~1,2 A típico (fonte de 2 A ou mais)
Placas	12 módulos + 1 backplane

1.1 Os módulos

Módulo	Pasta KiCad	Função
Clock	`clock-astavel-monoestavel`	Gera CLK e /CLK; automático (555 astável) ou manual (555 monoestável, sem repique); entrada HLT para parar
Reset	`reset-circuito`	Gera RST//RST (botão + reset ao ligar) e o sinal que zera o contador de passos
Contador de Programa (PC)	`pc-contador-programa`	Endereço da próxima instrução (0…15); incrementa (PCI), carrega (JMP), coloca no barramento (/PCO)
RAM e MAR	`ram-mar-memoria`	16 bytes de memória; MAR guarda o endereço; programação manual por chaves DIP
Registrador de Instrução (IR)	`ir-registrador-instrucao`	Guarda a instrução atual: 4 bits de opcode + 4 bits de operando
Decodificador de Controle	`cd-decodificador-controle`	Microcódigo em 3 EEPROMs: transforma (opcode, passo) em 18 sinais de controle
Contador de Passos	`fetch-contador-passos`	Conta os passos T0…T4 de cada instrução
Registrador A	`acc-registrador-a`	Acumulador: operando e resultado da ULA
Registrador B	`b-registrador-b`	Segundo operando da ULA
ULA	`alu-unidade-logica-aritmetica`	Soma, subtração, AND, OR, XOR e NOT de 8 bits
Registrador de Saída (OUT)	`out-registrador-saida`	Guarda o valor que vai para o display
Display Decimal	`display-decimal`	Mostra o OUT em decimal (EEPROM de conversão + varredura de 4 dígitos)
Backplane	`backplane-barramento`	Alimentação, barramento D0…D7 com LEDs e distribuição de todos os sinais de controle

1.2 Como uma instrução é executada

Todo ciclo de clock tem duas bordas:

- **Borda de subida de CLK:** os registradores selecionados copiam o que está no barramento.
- **Borda de descida de CLK** (subida de /CLK): o contador de passos avança e o microcódigo muda os sinais de controle para o próximo passo — assim os sinais estão estáveis antes da próxima borda de subida.

Os dois primeiros passos são iguais para todas as instruções (**busca**):

Passo	Sinais	O que acontece
T0	`PCO MAI`	o endereço do PC vai para o MAR
T1	`RMO IRI PCI`	a instrução lida da RAM vai para o IR; o PC é incrementado
T2...T4	dependem do opcode	**execução**

Exemplo — `ADD 15` (soma o conteúdo do endereço 15 ao A):

Passo	Sinais	O que acontece
T2	`IRO MAI`	o operando (15) vai do IR para o MAR
T3	`RMO BRI`	o valor da RAM[15] vai para o registrador B
T4	`ALU ACI`	o resultado A+B vai da ULA para o A

Depois de T4, o sinal /Y5 do 74LS138 zera o contador de passos (via placa de Reset) e a próxima instrução começa em T0.

1.3 Conjunto de instruções

Opcode	Binário	Mnemônico	Operando	Operação	Passos T2…T4
0	0000	NOP	—	não faz nada	—
1	0001	LDA end	endereço	A ← RAM[end]	IRO MAI / RMO ACI
2	0010	SUB end	endereço	A ← A − RAM[end]	IRO MAI / RMO BRI / ALU SUB ACI
3	0011	STA end	endereço	RAM[end] ← A	IRO MAI / ACO RMI
4	0100	ADD end	endereço	A ← A + RAM[end]	IRO MAI / RMO BRI / ALU ACI
5	0101	LDI n	valor 0…15	A ← n	IRO ACI
6	0110	JMP end	endereço	PC ← end	IRO JMP
7	0111	AND end	endereço	A ← A AND RAM[end]	IRO MAI / RMO BRI / ALU AL0 ACI
8	1000	OR end	endereço	A ← A OR RAM[end]	IRO MAI / RMO BRI / ALU AL1 ACI
9	1001	XOR end	endereço	A ← A XOR RAM[end]	IRO MAI / RMO BRI / ALU AL1 AL0 ACI
A	1010	NOT	—	A ← NOT A	ALU AL1 AL0 NOT ACI
B	1011	JC end	endereço	se carry: PC ← end	IRO JMP (só com CF = 1)
C	1100	JZ end	endereço	se zero: PC ← end	IRO JMP (só com ZF = 1)
D	1101	—	—	reservado (NOP)	—
E	1110	OUT	—	OUT ← A (display)	ACO ORI
F	1111	HLT	—	para o clock	HLT

Os opcodes originais do projeto (NOP = 0, LDA = 1, ADD = 4, OUT = 14, HLT = 15) foram mantidos. `JC` e `JZ` já estão no microcódigo e nas EEPROMs: enquanto não existir a placa de *flags*, as linhas CF/ZF ficam em 0 (resistores de pull-down no Decodificador) e as duas instruções se comportam como NOP.

1.4 Sinais de controle

O microcódigo é gravado em **lógica positiva** e os sinais que são **ativos em nível baixo** no hardware (entradas /OE do 74LS245, /G do 74LS173 e /LOAD do 74LS161) são invertidos na geração dos binários. Na tabela, a barra (`/`) indica nível ativo baixo.

EEPROM (chip)	IO7	IO6	IO5	IO4	IO3	IO2	IO1	IO0
EEPROM2 (U1)	HLT	PCI	/JMP	/PCO	/MAI	RMI	/RMO	/IRI
EEPROM1 (U2)	/IRO	/ACO	/ACI	SUB	AL1	AL0	/ALUO	NOT
EEPROM0 (U3)	/BRI	/ORI	livre (E0_5)	livre (E0_4)	—	—	—	—

Endereço das EEPROMs do microcódigo: `A8 = ZF, A7 = CF, A6…A3 = opcode, A2…A0 = passo`.

Tabela completa: quem gera e quem usa cada sinal

Sinal	Nivel ativo	Funcao	Origem	Destinos (placa: CI pino)
`CLK`	alto	clock do sistema	CLOCK: U3.8 (4Y)	PC: U1.2 (CLK) RAM: U6.7 (CLK) RAM: U11.4 (2A) RAM: U11.5 (2B) RAM: U11.9 (3A) RAM: U11.10 (3B) IR: U1.7 (CLK) IR: U2.7 (CLK) ACC: U1.7 (CLK) ACC: U2.7 (CLK) B: U1.7 (CLK) B: U2.7 (CLK) OUT: U1.7 (CLK) OUT: U2.7 (CLK)
`/CLK`	baixo	clock invertido (contador de passos)	CLOCK: U2.8 (3Y)	CLOCK: U3.9 (4A) FETCH: U1.2 (CLK)
`HLT`	alto	para o clock	CD: EEPROM2 IO7 (U1 pino 17)	CLOCK: U3.5 (3A)
`RST`	alto	reset (ativo alto, 74LS173)	RESET: U1.11 (4Y)	RESET: U1.9 (3A) RESET: U1.10 (3B) RAM: U6.15 (CLR) IR: U1.15 (CLR) IR: U2.15 (CLR) ACC: U1.15 (CLR) ACC: U2.15 (CLR) B: U1.15 (CLR) B: U2.15 (CLR) OUT: U1.15 (CLR) OUT: U2.15 (CLR)
`/RST`	baixo	reset (ativo baixo, 74LS161 do PC)	RESET: U1.8 (3Y)	RESET: U1.1 (1A) PC: U1.1 (/CLR)
`/Y5`	baixo	passo T5 alcancado	FETCH: U2.10 (/Y5)	RESET: U1.2 (1B)
`/STEPCLR`	baixo	zera o contador de passos	RESET: U1.6 (2Y)	FETCH: U1.1 (/CLR)

Sinal	Nivel ativo	Funcao	Origem	Destinos (placa: CI pino)
PCI	alto	incrementa o PC	CD: EEPROM2 IO6 (U1 pino 16)	PC: U1.7 (ENP) PC: U1.10 (ENT)
/JMP	baixo	carrega o PC (salto)	CD: EEPROM2 IO5 (U1 pino 15)	PC: U1.9 (/LOAD)
/PCO	baixo	PC -> barramento	CD: EEPROM2 IO4 (U1 pino 14)	PC: U2.19 (/OE)
/MAI	baixo	barramento -> MAR	CD: EEPROM2 IO3 (U1 pino 13)	RAM: U6.9 (/G1) RAM: U6.10 (/G2)
RMI	alto	barramento -> RAM (escrita)	CD: EEPROM2 IO2 (U1 pino 11)	RAM: U11.2 (1B)
/RMO	baixo	RAM -> barramento	CD: EEPROM2 IO1 (U1 pino 10)	RAM: U5.19 (/OE)
/IRI	baixo	barramento -> IR	CD: EEPROM2 IO0 (U1 pino 9)	IR: U1.9 (/G1) IR: U1.10 (/G2) IR: U2.9 (/G1) IR: U2.10 (/G2)
/IRO	baixo	operando do IR -> barramento	CD: EEPROM1 IO7 (U2 pino 17)	IR: U3.19 (/OE)
/ACI	baixo	barramento -> A	CD: EEPROM1 IO5 (U2 pino 15)	ACC: U1.9 (/G1) ACC: U1.10 (/G2) ACC: U2.9 (/G1) ACC: U2.10 (/G2)
/ACO	baixo	A -> barramento	CD: EEPROM1 IO6 (U2 pino 16)	ACC: U3.19 (/OE)
SUB	alto	subtracao na ULA	CD: EEPROM1 IO4 (U2 pino 14)	ALU: U18.1 (1A)
AL0	alto	selecao da ULA (bit 0)	CD: EEPROM1 IO2 (U2 pino 11)	ALU: U13.14 (A) ALU: U14.14 (A) ALU: U15.14 (A) ALU: U16.14 (A)
AL1	alto	selecao da ULA (bit 1)	CD: EEPROM1 IO3 (U2 pino 13)	ALU: U13.2 (B) ALU: U14.2 (B) ALU: U15.2 (B) ALU: U16.2 (B)
/ALUO	baixo	resultado da ULA -> barramento	CD: EEPROM1 IO1 (U2 pino 10)	ALU: U17.19 (/OE)
NOT	alto	inverte A (com AL=11)	CD: EEPROM1 IO0 (U2 pino 9)	ALU: U18.5 (3A)
/BRI	baixo	barramento -> B	CD: EEPROM0 IO7 (U3 pino 17)	B: U1.9 (/G1) B: U1.10 (/G2) B: U2.9 (/G1) B: U2.10 (/G2)
/ORI	baixo	barramento -> OUT	CD: EEPROM0 IO6 (U3 pino 16)	OUT: U1.9 (/G1) OUT: U1.10 (/G2) OUT: U2.9 (/G1) OUT: U2.10 (/G2)

1.5 Barramento e ligações diretas

- **Barramento D0…D7** (no backplane): PC, RAM, IR, A e ULA escrevem; MAR, RAM, IR, A, B e OUT leem. Só um módulo pode escrever por vez — o microcódigo garante isso (e o emulador em `firmware/ferramentas/emulador.py` verifica).
- **Ligações diretas** (cabo de 4 vias módulo a módulo, sem passar pelo backplane): A → ULA (A0…A7), B → ULA (B0…B7), OUT → Display (O0…O7), IR → Controle (opcode), Contador de Passos → Controle (T0…T2).

---

2. Revisão do projeto original e correções da versão 3.0

> Projeto de **Marcos Silva dos Santos** (MarcosObel) — GPL-3.0.

A versão 3.0 partiu das últimas versões em EasyEDA de cada módulo (pastas originais `1.0V`, `2.0V`, `2.1V`, `3.0V`, preservadas no Git na tag `v2-easyeda`). Todas as netlists originais foram lidas e comparadas automaticamente, pino a pino, com a nova descrição em KiCad. As placas **A, B, IR, OUT, Contador de Passos e Clock** ficaram eletricamente idênticas às originais; nas demais, as únicas diferenças são exatamente as correções abaixo.

2.1 Erros que impediam o computador de funcionar

#	Módulo	Problema encontrado	Correção na v3.0
1	**ULA** (1.0V)	As entradas de seleção **A/B (pinos 14 e 2) dos quatro 74LS153** — sinais AL0/AL1 — não estavam ligadas a nenhum conector. Entrada TTL solta = nível alto, então a ULA ficaria presa em "XOR" e **nunca somaria**.	AL0 e AL1 levados ao conector CTRL1 (pinos 3 e 4) e ao backplane.
2	**ULA** (1.0V)	As saídas **2Y de U5 e U6** (AND dos bits 1 e 5) estavam ligadas ao **GND**, junto com as entradas 2C1 de U13 e U15. Isso é um curto-circuito de saída sempre que o AND der 1, e a operação AND perdia os bits 1 e 5.	Redes `AND1` / `AND5` criadas: U5.6 → U13.11 e U6.6 → U15.11.
3	**Decodificador de Controle** (1.0V)	O pino **/WE (21) das três EEPROMs estava no GND**. O datasheet só garante leitura com /WE em nível alto; com /WE baixo a EEPROM fica fora do modo de leitura especificado e qualquer ruído em /OE pode disparar uma escrita (corrompendo o microcódigo).	/WE ligado ao +5 V.
4	**Microcódigo** (2021)	O código gravava **0 nos passos sem sinal**, mas /ACO, /IRO, /RMO, /ALUO, /PCO, /ACI, /BRI, /ORI, /MAI, /IRI e /JMP são **ativos em nível baixo**. Resultado: em todo passo "vazio" quatro módulos escreveriam no barramento ao mesmo tempo. O emulador mostra o conflito já no primeiro passo: `RMO, IRO, ACO, ALU`.	Microcódigo reescrito em lógica positiva com máscara de inversão por EEPROM (`ATIVO_BAIXO`).
5	**Reset** (1.0V)	O botão de reset usava um **resistor de 10 kΩ para o GND** numa entrada TTL. Uma entrada 74LS "puxa" ~0,4 mA para fora; com 10 kΩ a tensão fica acima de 1,4 V e é lida como **nível alto** → **reset permanentemente ativo**.	Botão ligado ao GND com pull-up de 10 kΩ (lógica do primeiro gate invertida); 74LS132 (Schmitt) recomendado; reset automático ao ligar; LED indicador. Placa de reset ganhou PCB (antes só tinha esquemático).
6	**Ordem do opcode**	O conector "REGIST." do IR tem o bit 7 no pino 1, mas o Decodificador ligava o pino 1 ao A3 (bit menos significativo do opcode). Com cabo reto, todos os opcodes chegariam **espelhados** (LDA = 0001 viraria 1000).	Conector OPCODE do Decodificador agora tem OP3 no pino 1 → cabo reto funciona.
7	**Display**	Não existia o conteúdo da EEPROM de conversão binário→decimal.	Gerador `firmware/display/display.py` (sem sinal e com sinal, zeros à esquerda apagados).
8	**Display**	O 74LS139 acende o dígito colocando o comum em **nível baixo** → o display precisa ser **cátodo comum**.	Especificado na lista de materiais e na serigrafia.
9	**Montagem** (pasta ASSEMBLY LAYOUT vazia)	Não havia nada interligando os módulos.	**Backplane** novo + tabela com os 52 cabos de 4 vias, verificada pino a pino.

2.2 Melhorias de confiabilidade

Módulo	Melhoria
ULA	**U18 74LS04 como buffer** de SUB e NOT: cada um desses sinais aciona 8–9 entradas TTL, mais do que a saída da EEPROM consegue drenar (≈2 mA).
ULA	**CARRY** (C4 do somador alto) disponível no conector CTRL2 para um futuro registrador de flags.
RAM	Pino THR (6) do 555 da chave PROG/RUN estava solto → ligado ao GND; CV com 10 nF.
RAM	Chaves DIP agora vão ao +5 V com redes de pull-down de 1 kΩ: **chave ligada = 1** (antes era invertido e as entradas ficavam flutuando).
RAM	Entradas não usadas do 74LS157 (U10) aterradas. Barramento entra e sai pelos mesmos 2 conectores (antes eram 5).
Contador de Passos	LEDs de passo agora acendem no passo **ativo** (antes acendiam os inativos) e ganharam resistores; entradas de carga do 74LS161 aterradas.
Contador de Programa	Alimentação em conector próprio (antes misturada com /PCO); o 74LS245 coloca 0 no nibble alto do barramento durante /PCO.
Clock	R1 de 150 Ω para 1 kΩ (corrente do pino de descarga do 556).
Decodificador	Linhas A7/A8 (CF/ZF) em conector FLAGS com pull-down → `JC` / `JZ` já funcionam quando a placa de flags existir.
Display	Resistores de 100 Ω em série com os segmentos; metade não usada do 74LS139 com entradas fixadas.
Todos	Desacoplamento de 100 nF ao lado de **cada** CI; PCBs de 2 camadas para fabricação profissional, com serigrafia didática (nome de cada sinal nos conectores e LEDs, part number e valor de cada componente).

2.3 Verificações automáticas feitas

1. **Netlist original × nova** (pino a pino, por CI) — só as diferenças intencionais acima.
2. **Esquemático KiCad × descrição** — a netlist exportada pelo `kicad-cli` de cada placa é comparada com `designs.py`: 0 diferenças.
3. **PCB** — DRC do KiCad sem erros e 0 ligações faltando em todas as placas (relatório em `hardware/fabricacao/<placa>/relatorio-drc.txt`).
4. **Cabos** — os 52 cabos (44 para o backplane + 8 diretos) foram verificados: cada pino liga o mesmo sinal nas duas pontas.
5. **Microcódigo** — o emulador em nível de sinais roda os programas de exemplo usando os **binários reais** das EEPROMs, verificando conflito de barramento em cada passo. As versões em Python e em C geram binários idênticos.

2.4 Checklist de revisão (atualiza o do README original)

- [x] Reviewer ACC REGISTER
- [x] Reviewer ARITHMETIC LOGIC UNIT
- [x] Reviewer ASSEMBLY LAYOUT (backplane novo)
- [x] Reviewer ASTABLE AND MONOSTABLE CLOCK CIRCUIT
- [x] Reviewer B REGISTER
- [x] Reviewer CONTROL DECODER
- [x] Reviewer DECIMAL DISPLAY
- [x] Reviewer FETCH EXECUTION COUNTER
- [x] Reviewer INSTRUCTION REGISTER
- [x] Reviewer OUTPUT REGISTER
- [x] Reviewer PROGRAM COUNTER
- [x] Reviewer RAM & MAR
- [x] Reviewer RESERT CIRCUIT

Os testes **em bancada** de cada placa produzida ainda serão registrados (o Clock 2.1V já foi testado).

3. Fabricação das placas (JLCPCB / PCBWay)

> Computador 8 Bits v3.0 — projeto de **Marcos Silva dos Santos** (MarcosObel). Licença GPL-3.0.

As 13 placas foram projetadas para **fabricação profissional** (2 camadas, furo metalizado) e, ao mesmo tempo, para serem **didáticas**: quem pega a placa na mão consegue seguir o circuito.

3.1 O que a placa mostra (pensado para estudo)

Na serigrafia (face de cima)	Para que serve
Nome do módulo e versão no topo	identificar a placa
Nome e pinagem de cada conector (ex.: `CONTROLE: CLK /ACI RST /ACO`)	saber o que entra e sai sem abrir o esquema
Part number de cada CI (74LS173, 74LS245…) e **valor** de resistores/capacitores	montar e estudar sem consultar a lista
Sinal que cada LED mostra (A7…A0, T0…T4, PROG/RUN…)	ver o computador "pensando"
Textos das chaves (END, DADO, GRAVAR, AUTO/MANUAL…)	operar o painel
Na face de baixo, em cobre: `8BITS <PLACA> v3.0 - M.S.SANTOS`	assinatura do projeto

Os componentes seguem a ordem do fluxo do sinal (conectores em cima → CIs → LEDs embaixo), o que deixa as trilhas fáceis de acompanhar com o esquemático (`0-esquematico.pdf`) ao lado.

Regras de projeto: 2 camadas · trilha 0,5 mm (alimentação 0,9 mm) · isolação 0,4 mm · via 1,6/0,8 mm · furos de 0,8 a 3,2 mm · 4 furos M3 de fixação · borda livre de 1,5 mm. Tudo dentro das capacidades padrão (as mais baratas) dos fabricantes.

3.2 Arquivos (em `hardware/fabricacao/<placa>/`)

Arquivo	Uso
`<placa>-gerber.zip`	**o arquivo que vai para a fábrica** (Gerber RS-274X + furação Excellon)
`0-esquematico.pdf`	diagrama elétrico da placa
`3-mascara-montagem-componentes.pdf`	guia de montagem em tamanho real: imprima e deixe ao lado durante a solda
`7-mapa-de-furacao.pdf`	diâmetros e posições dos furos
`lista-de-materiais.csv`	componentes da placa
`relatorio-drc.txt`	verificação de regras do KiCad (sem erros)
`1-…`, `2-…`, `4-…`, `5-…`, `6-…`	PDFs 1:1 das camadas de cobre e fotolitos — para estudo/conferência ou fabricação caseira (apêndice 3.5)

3.3 Pedido no JLCPCB (o PCBWay é praticamente igual)

1. Acesse **jlcpcb.com** → *Order now* → **Add gerber file** → escolha `hardware/fabricacao/<placa>/<placa>-gerber.zip`.
2. O site lê o tamanho e mostra a placa. Confira/ajuste:

Opção	Valor
Base Material	FR-4
Layers	**2**
PCB Qty	5 (o mínimo; sobra para presentear/estudar)
PCB Thickness	1.6 mm
PCB Color	à escolha (verde é a mais barata e rápida)
Surface Finish	HASL (with lead) ou LeadFree HASL
Outer Copper Weight	1 oz
Remove Order Number	*Specify a location* não é necessário — escolha "No" ou deixe o padrão
PCB Assembly	**desligado** (você mesmo solda)

3. *Save to cart* e repita para as 13 placas. Fazer **um único pedido** com todas sai mais barato no frete.
4. Pague e acompanhe. Prazo típico: alguns dias de produção + o frete escolhido.

> Dica: o backplane (≈ 205 × 150 mm) e a ULA (≈ 220 × 130 mm) passam de 100 × 100 mm, faixa em que alguns fabricantes cobram mais. As demais placas são pequenas.

Conferência antes de pedir (opcional)

Abra o `.zip` num visualizador de Gerber (o próprio site mostra; no Zorin dá para usar o `gerbv`: `sudo apt install gerbv`) e confira o contorno e os furos.

3.4 Quando as placas chegarem

1. Confira visualmente: trilhas íntegras, furos todos abertos, serigrafia legível.
2. Multímetro em continuidade entre **+5V e GND** do conector `ALIMENTACAO` (J1): **não pode apitar**.
3. Siga para 4. Montagem das placas.

3.5 Apêndice — fazer em casa (opcional)

Os PDFs `1-cobre-inferior-transferencia.pdf` e `2-cobre-superior-transferencia-espelhado.pdf` estão em **escala 1:1** (há uma régua de 50 mm em cada um — imprima em 100 %). Dá para fazer as placas pequenas (Reset, Clock, Passos, PC) por transferência térmica de toner em placa dupla face. Como os pads são nas duas faces e não há metalização, é preciso:

- fazer cada **via** com um pedaço de fio rígido soldado dos dois lados;
- soldar **também por cima** todo terminal que tenha trilha na face de cima (use soquetes torneados, que deixam espaço para o ferro).

Para as placas grandes, a fabricação profissional é muito mais confiável.

4. Montagem das placas — componente por componente

> Computador 8 Bits v3.0 — projeto de **Marcos Silva dos Santos** (MarcosObel). Licença GPL-3.0.

Antes de começar, tenha em mãos, para a placa que vai montar:

- o **guia de montagem** impresso (`hardware/fabricacao/<placa>/3-mascara-montagem-componentes.pdf`);
- a **lista de materiais** da placa (`lista-de-materiais.csv` na mesma pasta, ou lista completa);
- o **esquemático** (`0-esquematico.pdf`).

> 🧊 Para entender **o que cada placa faz** antes de montar — função, diagrama de funcionamento, papel de cada CI, significado dos LEDs, imagem 3D e pinagem — veja o guia das placas e o visualizador 3D.

4.1 Regras gerais (valem para todas as placas)

Como identificar a orientação

Componente	Marca na placa	No componente
CI / soquete	**pad quadrado = pino 1**; desenho com chanfro/meia-lua	chanfro ou bolinha marca o pino 1
LED	**pad quadrado = cátodo (K)**	perna **curta** / lado chato do LED = cátodo
Capacitor eletrolítico / tântalo	**pad quadrado = +**	faixa com "−" = negativo; tântalo: marca/faixa = **positivo**
Diodo (1N5822)	**pad quadrado = cátodo**	faixa = cátodo
Rede resistiva SIP	**pad quadrado = pino 1 (comum)**	ponto/marca = pino 1
Barra de pinos	pad quadrado = pino 1	—
Capacitor cerâmico/poliéster, resistor	sem polaridade	—

Ordem de soldagem (do mais baixo para o mais alto)

1. **Resistores** e **redes resistivas** (SIP).
2. **Diodo** (só o 1N5822 do backplane — faixa = cátodo).
3. **Soquetes de CI** — **use soquete em todos os CIs** (principalmente nas EEPROMs, que você vai regravar). Respeite o chanfro.
4. **Capacitores cerâmicos/poliéster** (100 nF de desacoplamento ao lado de cada CI).
5. **LEDs** (deixe-os todos na mesma altura: solde uma perna, ajuste, depois a outra).
6. **Capacitores eletrolíticos/tântalo** (polaridade!).
7. **Barras de pinos** (conectores de 4 pinos).
8. **Chaves, botões, potenciômetro, display.**
9. **CIs nos soquetes** — só depois do teste de alimentação sem CIs (seção 4.2).

Todas as soldas são feitas **pela face de baixo** (a face com o texto `8BITS ...` em cobre). O furo metalizado já liga as duas faces. Componentes ficam na face de cima, do lado da serigrafia.

Boas práticas de solda

- Ferro limpo e estanhado, 330–360 °C. Encoste a ponta no pad **e** no terminal, 1–2 s, aplique o estanho, retire o estanho e depois o ferro. A solda deve ficar brilhante, em forma de "vulcão".
- Corte as pernas rente (≈1 mm) depois de soldar.
- Nos soquetes, solde primeiro dois pinos em cantos opostos, confira se o soquete está assentado e só então solde o resto.
- Após cada placa, limpe o fluxo com álcool isopropílico e uma escova de dentes velha.

4.2 Teste de alimentação (fazer em TODAS as placas)

1. **Sem nenhum CI nos soquetes**, ligue 5 V no conector `ALIMENTACAO` (J1): pinos 1 e 2 = +5 V, pinos 3 e 4 = GND.
2. Meça 5 V entre o pino de VCC e o pino de GND de **cada soquete**:

Encapsulamento	VCC	GND
DIP-8 (555)	8	1
DIP-14 (74LS00/04/08/32/86/132, NE556)	14	7
DIP-16 (74LS138/139/153/157/161/173/283, 74S189)	16	8
DIP-20 (74LS245)	20	10
DIP-24 (28C16)	24	12

3. Desligue, insira os CIs (orientação!) e ligue de novo, de preferência com uma fonte com limite de corrente (~500 mA por placa). Se algum CI esquentar, desligue imediatamente e procure CI invertido ou curto.

> Dica: para os testes individuais abaixo, use jumpers macho-fêmea nas barras de pinos. "Nível alto" = ligar ao +5 V; "nível baixo" = ligar ao GND. **Entrada TTL desconectada é lida como nível ALTO.**

4.3 Montagem e teste de cada placa

Ordem recomendada de montagem (cada placa testada antes de seguir): **Backplane → Clock → Reset → Contador de Passos → PC → RAM/MAR → IR → Controle → A → B → ULA → Saída → Display.**

Backplane / Barramento (`backplane-barramento`)

- **O que tem:** entrada de 5 V (jack J1 *ou* borne J2), chave liga/desliga SW1, fusível rearmável F1 (PTC 3 A), diodo de proteção D1, capacitores C1/C2, LED de energia, 8 LEDs do barramento (D7…D0) e 45 conectores de 4 pinos.
- **Montagem:** resistores R1–R9 → C2 → LEDs → C1 (polaridade) → D1 (faixa no pad quadrado) → F1 → SW1 → J1/J2 → todos os conectores (pino 1 = pad quadrado, à esquerda).
- **Proteção contra inversão:** se a fonte for ligada invertida, D1 conduz e o PTC F1 abre. Desligue, corrija e espere esfriar.
- **Teste:** ligue a fonte de 5 V; LED1 acende; meça 5 V nos pinos 1–2 de cada conector `P_xxx`. Os LEDs do barramento ficam apagados (barramento solto).

Clock astável/monoestável (`clock-astavel-monoestavel`)

- **CIs:** U1 NE556, U2 74LS00, U3 74LS04.
- **Atenção:** C1 é **tântalo 10 µF** (polaridade: o lado marcado do tântalo é o **positivo**). R1 agora é 1 kΩ (era 150 Ω).
- **Montagem:** R1, R2 (10 Ω), R4 (10 k), R5 (330 k), R6 → soquetes → C2, C3 (10 nF), C4 (330 nF), C5–C7 (100 nF) → C1 → LED1 → J1, J2 → RV1 (potenciômetro) → SW1 (PASSO) → SW2 (AUTO/MANUAL).
- **Teste:** ligue o pino 3 de J2 (HLT) ao **GND** com um jumper (HLT solto = relógio parado!). Em AUTO, LED1 pisca e o potenciômetro muda a velocidade (~0,7 Hz a ~140 Hz). Em MANUAL, cada toque em PASSO dá um pulso. Ligue HLT ao +5 V: o clock para.

Reset (`reset-circuito`)

- **CI:** U1 **74LS132** (NAND Schmitt; um 74LS00 também funciona).
- **Montagem:** R1, R2 (10 k), R3 (1 k) → soquete → C2 → LED1 → C1 10 µF (polaridade) → J1, J2 → SW1.
- **Teste:** ao ligar, LED1 pisca (reset automático de ~0,1 s). Apertar RESET: LED1 acende, J2.1 (RST) = 5 V e J2.2 (/RST) = 0 V. J2.4 (/STCLR) vai a 0 V durante o reset **ou** quando J2.3 (/Y5) é aterrado.

Contador de Passos (`fetch-contador-passos`)

- **CIs:** U1 74LS161, U2 74LS138.
- **Montagem:** R1–R3 (330 Ω), R4–R8 (470 Ω) → soquetes → C1, C2 → LED1–LED3 (amarelos, binário T2 T1 T0) → LED4–LED8 (passos T0…T4) → J1–J3.
- **Teste (sem a placa de reset):** /STCLR (J3.3) em +5 V, pulse /CLK (J3.2). Os LEDs de passo andam T0→T7 (só T0–T4 têm LED). Agora ligue J3.1 (/Y5) em J3.3 (/STCLR) com um jumper: o contador passa a voltar para T0 depois de T4.

Contador de Programa — PC (`pc-contador-programa`)

- **CIs:** U1 74LS161, U2 74LS245.
- **Montagem:** R1–R4 → soquetes → C1, C2 → LED1–LED4 (P3…P0) → J1–J5.
- **Teste:** /RST (J2.4) e /JMP (J2.3) em +5 V, PCI (J2.1) em +5 V. A cada pulso em CLK (J2.2) os LEDs contam 0…15. PCI em GND: para de contar. /PCO (J3.1) em GND: o valor aparece em BUS_L (J4); BUS_H (J5) fica em 0.

RAM e MAR (`ram-mar-memoria`)

- **CIs:** U1, U2 74S189 · U3, U4 74LS04 · U5 74LS245 · U6 74LS173 (MAR) · U7–U10 74LS157 · U11 74LS00 · U12 NE555.
- **Montagem:** resistores R1–R18 → RN1 (SIP-9) e RN2 (SIP-5) com o **pino 1 (comum) no pad quadrado** → soquetes → C1–C12 (100 nF), C13 (1 nF), C14 (10 nF) → LEDs (verdes = dado, amarelos = endereço, vermelho LED13 = PROG, verde LED14 = RUN) → J1–J5 → SW1 (DIP 4), SW3 (DIP 8), SW4 (GRAVAR), SW2 (PROG/RUN).
- **Chaves DIP (mudança da v3.0):** chave **LIGADA = 1**. A chave 1 é o bit 0.
- **Teste de programação manual:** SW2 em PROG (LED vermelho). Escolha o endereço em SW1 e o dado em SW3, aperte GRAVAR: os LEDs verdes mostram o dado gravado. Mude o endereço e volte: o dado continua lá. Ligue /RMO (J3.3) ao GND e veja o dado em BUS_L/BUS_H.

Registrador de Instrução — IR (`ir-registrador-instrucao`)

- **CIs:** U1, U2 74LS173, U3 74LS245.
- **Montagem:** R1–R8 → soquetes → C1–C3 → LED1–LED8 (LED1–4 = opcode, LED5–8 = operando) → J1–J5.
- **Teste:** coloque um valor em BUS_L/BUS_H com jumpers, /IRI (J2.2) em GND e dê um pulso de CLK: os LEDs mostram o valor. RST (J2.3) em +5 V apaga. Com /IRO (J2.4) em GND, só o **nibble baixo** volta ao barramento (o alto sai como 0).

Decodificador de Controle (`cd-decodificador-controle`)

- **CIs:** U1 = EEPROM2.bin, U2 = EEPROM1.bin, U3 = EEPROM0.bin (28C16 / AT28C16). **Grave antes** (6. Gravação) e marque cada chip com uma etiqueta.
- **Correção crítica da v3.0:** o pino 21 (/WE) agora vai ao +5 V. Na placa antiga ele estava no GND.
- **Montagem:** R1, R2 (10 k) → soquetes de 24 pinos **largos (600 mil)** → C1–C3 → J1–J9.
- **Teste:** com PASSO (J2) e OPCODE (J3) todos em GND (instrução NOP, passo T0), os sinais ativos devem ser apenas `/PCO` e `/MAI` (nível **baixo** em J6.1 e J5.4); os demais sinais ativos-baixos ficam em 5 V e os ativos-altos em 0 V. Use a tabela de sinais em 1. Arquitetura.

Registrador A (`acc-registrador-a`) e Registrador B (`b-registrador-b`)

- **CIs:** U1, U2 74LS173, U3 74LS245 (as duas placas são idênticas, mudam só os nomes dos sinais).
- **Montagem:** R1–R8 → soquetes → C1–C3 → LED1–LED8 (LED1 = bit 7, à esquerda) → J1–J6.

- **Teste:** igual ao IR, com /ACI (ou /BRI) e /ACO. As saídas ALU_L/ALU_H (J5/J6) mostram sempre o conteúdo.

Unidade Lógica e Aritmética — ULA (`alu-unidade-logica-aritmetica`)

- **CIs:** U1, U2 74LS283 · U3, U4, U10, U12 74LS86 · U5, U6 74LS08 · U7, U8, U9, U11 74LS32 · U13–U16 74LS153 · U17 74LS245 · U18 74LS04 (novo, buffer de SUB/NOT).
- **Atenção:** é a placa com mais CIs (18). Confira **duas vezes** o tipo de cada CI antes de inserir — 74LS08, 74LS32 e 74LS86 têm a mesma pinagem e são fáceis de confundir.
- **Montagem:** R1–R8 → 18 soquetes → C1–C18 → LED1–LED8 → J1–J9.
- **Teste (com A e B ligadas pelos cabos diretos, ou com jumpers):**

AL1	AL0	SUB	NOT	Resultado nos LEDs
0	0	0	0	A + B
0	0	1	0	A − B
0	1	0	0	A AND B
1	0	0	0	A OR B
1	1	0	0	A XOR B
1	1	0	1	NOT A

Registrador de Saída — OUT (`out-registrador-saida`)

- **CIs:** U1, U2 74LS173. Sem 74LS245 (não escreve no barramento).
- **Montagem:** R1–R8 → soquetes → C1, C2 → LEDs → J1–J6.
- **Teste:** como o registrador A, usando /ORI.

Display Decimal (`display-decimal`)

- **CIs:** U1 = DISPLAY.bin (28C16), U2 NE555, U3 74LS161, U4 74LS139. **DS1 = display de 4 dígitos 0,4" CÁTODO COMUM** (12 pinos).
- **Atenção:** o display tem de ser **cátodo comum** (ex.: 5641AS / FJ4401AH / CC 0,4"). Um de ânodo comum não acende com este circuito. Confira a pinagem do seu display no datasheet: 1=e 2=d 3=dp 4=c 5=g 6=DIG4 7=b 8=DIG3 9=DIG2 10=f 11=a 12=DIG1, fileiras distantes 12,7 mm.
- **Montagem:** R1 (1 k), R10 (100 k), R2–R9 (100 Ω, segmentos) → soquetes → C1–C6 → J1–J3 → SW1 → DS1 (pode usar barra de pinos fêmea como soquete).
- **Teste:** com as entradas (J2/J3) em GND o display mostra **0**; com todas em +5 V mostra **255** (chave em "sem sinal") ou **-1** (chave em "com sinal").

5. Interligação: backplane, cabos e montagem na base

> Projeto de **Marcos Silva dos Santos** (MarcosObel) — GPL-3.0.

5.1 Ideia geral

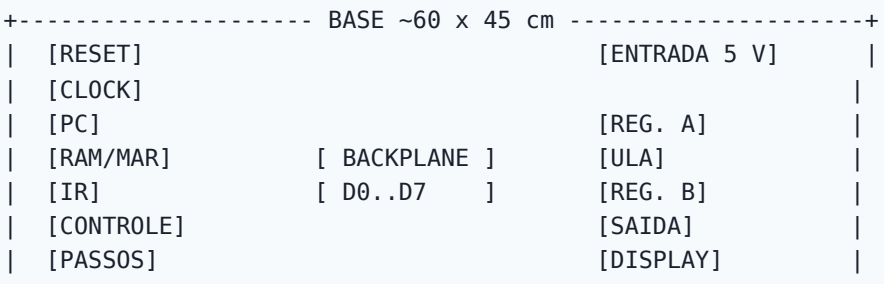
Cada módulo tem conectores de 4 pinos (barra de pinos 2,54 mm). O **backplane** (pasta `backplane-barramento`, antiga "ASSEMBLY LAYOUT") é a placa-mãe: recebe os 5 V, distribui a alimentação, contém o **barramento D0…D7** (com 8 LEDs) e roteia **todos** os sinais de controle.

Regras simples:

1. **Todos os cabos são de 4 vias e RETOS** — pino 1 de uma ponta no pino 1 da outra (fio da mesma cor sempre no pino 1).
2. No backplane, os conectores estão **agrupados por módulo**, com o nome do módulo ao lado (CLOCK, RESET, PC, RAM/MAR, IR, CONTROLE, PASSOS, REG. A, ULA, REG. B, SAÍDA, DISPLAY).
3. Cada conector tem o mesmo nome e a mesma ordem de sinais do conector correspondente no módulo (impressos na serigrafia, embaixo de cada pino).
4. São **52 cabos**: 44 vão do módulo ao backplane e 8 são **diretos** entre módulos vizinhos.

5.2 Arranjo físico sugerido

Monte os módulos numa base de MDF/acrílico (~60 × 45 cm) com espaçadores M3 de 10 mm, no mesmo arranjo do diagrama de blocos — assim todos os cabos de 20 cm alcançam:



- O **Controle** fica logo abaixo do **IR** (cabo direto do opcode) e acima de **Passos** (cabo direto T0…T2).
- A **ULA** fica entre **A** e **B** (dois cabos diretos de cada lado); o **Display** abaixo da **Saída**.

5.3 Alimentação

- Use uma **fonte regulada de 5 V, 2 A ou mais** (3 A recomendado), plugue P4 5,5 × 2,1 mm, **centro positivo** — ou ligue os fios no borne J2.
- Consumo típico do computador completo: **1,0 a 1,4 A** (depende de quantos LEDs estão acesos).
- **Não use fonte de 9 V/12 V**: não há regulador. O diodo D1 + PTC F1 protegem apenas contra inversão de polaridade e curto.
- Cada cabo de alimentação leva +5 V em 2 fios e GND em 2 fios, então a queda de tensão é desprezível.

5.4 Tabela de cabos

#	Placa	Conector	Sinais (pino 1 -> 4)	Liga em
1	Clock	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J12** (P_CLOCK)
2	Clock	J2 CLOCK	`CLK /CLK HLT NC`	Backplane **J45** (C_CLOCK)
3	Reset	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J13** (P_RESET)
4	Reset	J2 RESET	`RST /RST /Y5 /STCLR`	Backplane **J46** (C_RESET)
5	Contador de Programa	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J8** (P_PC)
6	Contador de Programa	J2 CONTROLE	`PCI CLK /JMP /RST`	Backplane **J33** (C_PC1)
7	Contador de Programa	J3 CONTROLE2	`/PCO NC NC NC`	Backplane **J34** (C_PC2)
8	Contador de Programa	J4 BUS_L	`D0 D1 D2 D3`	Backplane **J20** (BL_PC)
9	Contador de Programa	J5 BUS_H	`D4 D5 D6 D7`	Backplane **J27** (BH_PC)
10	RAM e MAR	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J9** (P_RAM)
11	RAM e MAR	J2 CONTROLE1	`CLK /MAI RST RMI`	Backplane **J35** (C_RAM1)
12	RAM e MAR	J3 CONTROLE2	`/WE /CLK /RMO RUN`	Backplane **J36** (C_RAM2)
13	RAM e MAR	J4 BUS_L	`D0 D1 D2 D3`	Backplane **J21** (BL_RAM)
14	RAM e MAR	J5 BUS_H	`D4 D5 D6 D7`	Backplane **J28** (BH_RAM)
15	Reg. Instrucao	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J6** (P_IR)
16	Reg. Instrucao	J2 CONTROLE	`CLK /IRI RST /IRO`	Backplane **J31** (C_IR)
17	Reg. Instrucao	J3 BUS_L	`D0 D1 D2 D3`	Backplane **J18** (BL_IR)
18	Reg. Instrucao	J4 BUS_H	`D4 D5 D6 D7`	Backplane **J25** (BH_IR)
19	Reg. Instrucao	J5 OPCODE	`OP3 OP2 OP1 OP0`	**cabo direto** -> Decodificador de Controle J3 (OPCODE)
20	Decodificador de Controle	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J10** (P_CD)
21	Decodificador de Controle	J5 CTRL_A	`/IRI /RMO RMI /MAI`	Backplane **J39** (CD_A)
22	Decodificador de Controle	J6 CTRL_B	`/PCO /JMP PCI HLT`	Backplane **J40** (CD_B)
23	Decodificador de Controle	J7 CTRL_C	`NOT /ALUO AL0 AL1`	Backplane **J41** (CD_C)
24	Decodificador de Controle	J8 CTRL_D	`SUB /ACI /ACO /IRO`	Backplane **J42** (CD_D)

#	Placa	Conector	Sinais (pino 1 -> 4)	Liga em
25	Decodificador de Controle	J9 CTRL_E	`/BRI /ORI E0_5 E0_4`	Backplane **J43** (CD_E)
26	Contador de Passos	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J11** (P_FETCH)
27	Contador de Passos	J2 PASSO	`T0 T1 T2 NC`	**cabo direto** -> Decodificador de Controle J2 (PASSO)
28	Contador de Passos	J3 CONTROLE	`/Y5 /CLK /STCLR NC`	Backplane **J44** (C_FETCH)
29	Registrador A	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J3** (P_ACC)
30	Registrador A	J2 CONTROLE	`CLK /ACI RST /ACO`	Backplane **J29** (C_ACC)
31	Registrador A	J3 BUS_L	`D0 D1 D2 D3`	Backplane **J15** (BL_ACC)
32	Registrador A	J4 BUS_H	`D4 D5 D6 D7`	Backplane **J22** (BH_ACC)
33	Registrador A	J5 ALU_L	`A0 A1 A2 A3`	**cabo direto** -> ULA J4 (ACC_L)
34	Registrador A	J6 ALU_H	`A4 A5 A6 A7`	**cabo direto** -> ULA J5 (ACC_H)
35	Registrador B	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J4** (P_B)
36	Registrador B	J2 CONTROLE	`CLK /BRI RST /BRO`	Backplane **J30** (C_B)
37	Registrador B	J3 BUS_L	`D0 D1 D2 D3`	Backplane **J16** (BL_B)
38	Registrador B	J4 BUS_H	`D4 D5 D6 D7`	Backplane **J23** (BH_B)
39	Registrador B	J5 ALU_L	`B0 B1 B2 B3`	**cabo direto** -> ULA J6 (B_L)
40	Registrador B	J6 ALU_H	`B4 B5 B6 B7`	**cabo direto** -> ULA J7 (B_H)
41	ULA	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J5** (P_ALU)
42	ULA	J2 BUS_L	`D0 D1 D2 D3`	Backplane **J17** (BL_ALU)
43	ULA	J3 BUS_H	`D4 D5 D6 D7`	Backplane **J24** (BH_ALU)
44	ULA	J8 CTRL1	`/ALUO SUB AL0 AL1`	Backplane **J37** (C_ALU1)
45	ULA	J9 CTRL2	`NOT CARRY NC NC`	Backplane **J38** (C_ALU2)
46	Registrador de Saida	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J7** (P_OUT)
47	Registrador de Saida	J2 CONTROLE	`CLK /ORI RST NC`	Backplane **J32** (C_OUT)
48	Registrador de Saida	J3 BUS_L	`D0 D1 D2 D3`	Backplane **J19** (BL_OUT)
49	Registrador de Saida	J4 BUS_H	`D4 D5 D6 D7`	Backplane **J26** (BH_OUT)
50	Registrador de Saida	J5 DISPLAY_L	`O0 O1 O2 O3`	**cabo direto** -> Display Decimal J2 (ENTRADA_L)
51	Registrador de Saida	J6 DISPLAY_H	`O4 O5 O6 O7`	**cabo direto** -> Display Decimal J3 (ENTRADA_H)
52	Display Decimal	J1 ALIMENTACAO	`+5V +5V GND GND`	Backplane **J14** (P_DISPLAY)

Pinos que mudam de nome de propósito

- **Reg. B J2 pino 4 (/BRO):** no backplane vai fixo em +5 V — o registrador B nunca escreve no barramento.
- **RAM J3 pinos 1, 2 e 4 (/WE, /CLK, RUN):** são **saídas de teste** da placa de RAM; no backplane esses pinos não estão ligados.
- **ULA J9 pino 2 (CARRY):** reservado para o futuro registrador de flags (não ligado no backplane).
- **Controle J4 (FLAGS):** fica livre até existir a placa de flags (pull-down interno = flags em 0).
- **Backplane EXPANSAO:** E0_5, E0_4 (saídas livres da EEPROM0), CLK e RST para futuras extensões.

5.5 Ordem de ligação recomendada

1. Backplane sozinho → confira os 5 V e o LED de energia.
2. Clock + Reset → observe o LED do clock e o reset.
3. Passos → os LEDs T0…T4 devem andar com o clock (com o Reset ligado, voltam a T0 depois de T4).
4. PC, RAM/MAR, IR, Controle (com as EEPROMs gravadas).
5. A, B, ULA, Saída, Display.
6. Programe a RAM e rode o primeiro programa (veja 7. Programação e uso).

6. Gravação das EEPROMs (no Zorin OS / Ubuntu)

> Projeto de **Marcos Silva dos Santos** (MarcosObel) — GPL-3.0.

O computador usa **4 EEPROMs 28C16** (AT28C16, CAT28C16A ou equivalente, 2 KB, DIP-24 largo):

Arquivo	Chip	Placa	Conteúdo
`firmware/microcodigo/EEPROM2.bin`	**U1**	Decodificador de Controle	HLT PCI /JMP /PCO /MAI RMI /RMO /IRI
`firmware/microcodigo/EEPROM1.bin`	**U2**	Decodificador de Controle	/IRO /ACO /ACI SUB AL1 AL0 /ALUO NOT
`firmware/microcodigo/EEPROM0.bin`	**U3**	Decodificador de Controle	/BRI /ORI (+ 6 bits livres)
`firmware/display/DISPLAY.bin`	**U1**	Display Decimal	tabela binário → 7 segmentos

Cada arquivo tem exatamente 2048 bytes. Também há versões `.hex` (Intel HEX) para gravadores que preferem esse formato.

> **Etiquete cada chip** (fita crepe: "CD-U1 EEPROM2", "CD-U2 EEPROM1", "CD-U3 EEPROM0", "DISPLAY"). Trocar dois chips de lugar faz o computador "enlouquecer" sem nenhum dano — mas é difícil de achar.

6.1 Gerar (ou regerar) os binários

Os binários já vêm prontos no repositório. Para regerar depois de mudar o microcódigo:

```
cd firmware/microcodigo
python3 microcodigo.py          # gera EEPROM0/1/2.bin e .hex e imprime a tabela de instruções
cd ../display
python3 display.py              # gera DISPLAY.bin e .hex
```

Versão em C (mesma saída, byte a byte):

```
cd firmware/microcodigo/c
make bins                        # compila com gcc e gera EEPROM0/1/2.bin
```

6.2 Gravar com TL866II Plus / XGecu T48 usando `minipro`

1. Instale o `minipro` (programa livre para os gravadores TL866/T48):

   ```
   sudo apt update
   sudo apt install minipro        # disponível no Ubuntu 23.04+ / Debian 12+
   ```

   Se o pacote não existir na sua versão do Zorin, compile:

   ```
   sudo apt install build-essential pkg-config git libusb-1.0-0-dev zlib1g-dev
   git clone https://gitlab.com/DavidGriffith/minipro.git
   cd minipro && make && sudo make install
   sudo cp udev/*.rules /etc/udev/rules.d/ && sudo udevadm control --reload-rules && sudo
   udevadm trigger
   ```

2. Encontre o nome exato do seu chip na lista do `minipro`:

   ```
   minipro -l | grep -i 28C16
   ```

3. Coloque a EEPROM no soquete ZIF (pino 1 do lado da alavanca, alinhada embaixo — veja a figura do gravador) e grave:

   ```
   minipro -p "AT28C16" -w firmware/microcodigo/EEPROM2.bin      # chip U1 do Controle
   minipro -p "AT28C16" -w firmware/microcodigo/EEPROM1.bin      # chip U2 do Controle
   minipro -p "AT28C16" -w firmware/microcodigo/EEPROM0.bin      # chip U3 do Controle
   minipro -p "AT28C16" -w firmware/display/DISPLAY.bin          # chip U1 do Display
   ```

   O `minipro` grava e confere automaticamente. Para conferir de novo depois:

   ```
   minipro -p "AT28C16" -m firmware/microcodigo/EEPROM2.bin
   ```

   Se aparecer erro de ID do chip, acrescente `-y` (as 28C16 não têm ID eletrônico).

6.3 Sem gravador?

Dá para gravar uma 28C16 com um Arduino e dois 74HC595 (o "EEPROM programmer" popularizado pelo Ben Eater). Use os arquivos `.bin` deste repositório como dados. Lembre-se: o computador em si continua sem nenhum microcontrolador — o Arduino só serve como ferramenta de gravação.

6.4 Conferir sem o computador montado

O emulador lê os mesmos `.bin` e executa um programa sinal a sinal, acusando qualquer conflito no barramento:

```
cd firmware
python3 ferramentas/emulador.py programas/01-soma.asm            # resultado: [42]
python3 ferramentas/emulador.py programas/04-fibonacci.asm --max 400
python3 ferramentas/emulador.py programas/01-soma.asm --passos   # mostra cada passo T0..T4
```

7. Programação e uso

> Projeto de **Marcos Silva dos Santos** (MarcosObel) — GPL-3.0.

7.1 Painel de controle (onde fica cada coisa)

Placa	Controle	Função
Backplane	SW1	liga/desliga
Reset	RESET	zera PC, MAR, A, B, IR, OUT e o contador de passos (o computador também se reseta sozinho ao ligar)
Clock	AUTO / MANUAL	clock automático (LED pisca) ou passo a passo
Clock	PASSO	um pulso de clock por toque (modo MANUAL)
Clock	VELOCIDADE	potenciômetro: ~0,7 Hz a ~140 Hz
RAM	PROG / RUN	PROG (LED vermelho) = você grava pelas chaves · RUN (LED verde) = o computador usa a RAM
RAM	END (DIP 4)	endereço a gravar (A3 A2 A1 A0)
RAM	DADO (DIP 8)	byte a gravar (D7 … D0) — **chave ligada = 1**
RAM	GRAVAR	grava o DADO no END (só em PROG)
Display	COM SINAL / SEM SINAL	mostra 0…255 ou −128…127

7.2 Escrever um programa

Programas ficam em `firmware/programas/*.asm`. Exemplo (`01-soma.asm`):

```
; Soma 28 + 14 = 42 e mostra no display.
        LDA x       ; A <- 28
        ADD y       ; A <- A + 14
        OUT         ; display = 42
        HLT
x:      DB 28
y:      DB 14
```

- Cada linha ocupa **1 byte** (16 bytes no total): instrução + operando de 4 bits, ou `DB` (dado de 8 bits).
- Rótulos (`x:`) viram o endereço da linha. Operandos aceitam decimal, `0x0F`, `0b1010` ou rótulo.

Montar (gera a tabela das chaves)

```
cd firmware
python3 ferramentas/montador.py programas/01-soma.asm
```

Saída:

```
End  Chaves-END  Chaves-DADO   Hex  Binario    Instrucao
     (4 3 2 1)   (8 7 6 5 4 3 2 1)
 0   ····        ···█·█··     14   00010100   LDA x
 1   ···█        ·█···█·█     45   01000101   ADD y
 2   ··█·        ██·····      E0   11100000   OUT
 3   ··██        ████····     F0   11110000   HLT
 4   ·█··        ···██··      1C   00011100   DB 28
 5   ·█·█        ····███·     0E   00001110   DB 14
█ = chave LIGADA (1)   · = chave DESLIGADA (0).  Chave 1 = bit 0 (direita).
```

Testar no emulador antes de gravar

```
python3 ferramentas/emulador.py programas/01-soma.asm            # Saidas no display: [42]
python3 ferramentas/emulador.py programas/01-soma.asm --passos   # cada passo, sinais e
barramento
```

7.3 Gravar o programa na RAM (pelas chaves)

1. Clock em **MANUAL** (assim nada roda enquanto você grava).
2. RAM em **PROG** (LED vermelho aceso). Os LEDs amarelos mostram o endereço das chaves END e os verdes mostram o conteúdo da RAM nesse endereço.
3. Para cada linha da tabela do montador:
    1. ajuste **END** (chaves 4 3 2 1 = A3 A2 A1 A0);
    2. ajuste **DADO** (chaves 8 … 1 = D7 … D0);
    3. aperte **GRAVAR** — os LEDs verdes passam a mostrar o dado.
4. Confira passando de novo pelos endereços (sem apertar GRAVAR).

> A RAM 74S189 é volátil: o programa se perde ao desligar.

7.4 Executar

1. RAM em **RUN** (LED verde).
2. Aperte **RESET**.
3. Clock em **AUTO** (ou vá dando PASSO para acompanhar cada microinstrução nos LEDs).
4. O resultado aparece no display; `HLT` para o clock.

Acompanhe nos LEDs: Passos (T0…T4), PC, endereço da RAM, IR (opcode/operando), barramento (backplane), A, B, ULA e OUT.

7.5 Programas de exemplo (`firmware/programas/`)

Arquivo	O que faz	Resultado no display
`01-soma.asm`	28 + 14	42
`02-subtracao.asm`	100 − 58	42
`03-contador.asm`	conta 0, 1, 2 … 255, 0 …	contagem infinita
`04-fibonacci.asm`	sequência de Fibonacci (usa STA, 16 bytes)	0 1 1 2 3 5 8 13 21 34 55 89 144 233 …
`05-logica.asm`	AND, OR, XOR, NOT de 0xF0 e 0x3C	48, 252, 204, 15
`06-multiplica-por-soma.asm`	tabuada do 7 por soma repetida	7, 14, 21, 28 …

Todos foram verificados no emulador com os binários finais das EEPROMs.

7.6 Conjunto de instruções (resumo)

Hex	Instrução	Operação
0	`NOP`	nada
1	`LDA end`	A ← RAM[end]
2	`SUB end`	A ← A − RAM[end]
3	`STA end`	RAM[end] ← A
4	`ADD end`	A ← A + RAM[end]
5	`LDI n`	A ← n (0…15)
6	`JMP end`	PC ← end
7	`AND end`	A ← A AND RAM[end]
8	`OR end`	A ← A OR RAM[end]
9	`XOR end`	A ← A XOR RAM[end]
A	`NOT`	A ← NOT A
B	`JC end`	salta se carry (requer placa de flags)
C	`JZ end`	salta se zero (requer placa de flags)
E	`OUT`	display ← A
F	`HLT`	para

Detalhes de cada microinstrução: 1. Arquitetura.

8. Testes e diagnóstico

> Projeto de **Marcos Silva dos Santos** (MarcosObel) — GPL-3.0.

8.1 Ferramentas

- Multímetro (tensão e continuidade).
- **Ponta de prova lógica** simples: um LED + resistor de 1 kΩ num jumper (acende = nível alto).
- Jumpers macho-fêmea para forçar níveis (+5 V / GND) nas barras de pinos.
- Clock em **MANUAL**: o computador pode ser executado um passo por vez, indefinidamente devagar.
- O **emulador** (`firmware/ferramentas/emulador.py --passos`) mostra, passo a passo, o que cada LED deveria mostrar — compare com a placa real.

8.2 Subida do sistema (bring-up) em etapas

Etapa	Ligue	O que deve acontecer
1	Backplane	LED de energia aceso; 5,0 V (4,75–5,25 V) em todos os `P_xxx`
2	+ Clock (HLT ligado ao backplane)	LED do clock pisca em AUTO; com o Controle ainda sem EEPROMs, HLT fica solto e o clock **para** — normal
3	+ Reset + Passos	LEDs T0…T4 andam e voltam a T0; RESET leva a T0
4	+ PC	PC só conta com PCI; teste forçando PCI = 1 com jumper no conector C_PC1 do backplane
5	+ RAM/MAR	grave e leia valores em PROG (seção 7.3)
6	+ IR + Controle (EEPROMs gravadas)	em T0/T1 o IR recebe o byte da RAM; o PC incrementa em T1
7	+ A, B, ULA, Saída, Display	rode `01-soma.asm` passo a passo e compare com o emulador

8.3 Problemas comuns

Sintoma	Causa provável	O que fazer
Nada acende; PTC quente	fonte invertida ou curto	desligue, confira polaridade (centro +), procure curto entre +5 V e GND
Clock não pisca	HLT solto (nível alto = parado)	ligue o cabo C_CLOCK ao backplane (HLT vem do Controle) ou aterre HLT para testar
Clock "pula" passos no MANUAL	toque longo no botão PASSO	o monoestável dá ~0,12 s por pulso; solte o botão entre toques
Passos não voltam a T0	cabo C_FETCH ou C_RESET invertido; /Y5 não chega à placa de Reset	confira os dois cabos (pino 1 com pino 1)
Reset não funciona / LED de reset sempre aceso	botão invertido, C1 invertido	confira C1 (polaridade) e SW1
Barramento com valor "estranho", CIs 74LS245 quentes	dois módulos escrevendo ao mesmo tempo	EEPROMs trocadas (U1/U2/U3) ou gravadas com o microcódigo antigo; regrave com os `.bin` da v3.0
Instruções erradas (ex.: LDA age como OR)	opcode chegando invertido	o cabo IR→Controle deve ser reto (OP3 no pino 1 dos dois lados)
ULA só faz XOR	AL0/AL1 não chegam	placa ULA antiga (1.0V) ou cabo C_ALU1 desligado
Subtração errada no bit 0	carry de entrada (SUB) não chega ao 74LS283	confira U18 (buffer) e o cabo C_ALU1
RAM grava sozinha / perde dados	RMI ruidoso, C13/R18 do pulso de escrita	confira C13 = 1 nF e R18 = 1 kΩ; desacoplamento perto do 74S189
Chaves DIP sem efeito	RN1/RN2 invertidas (pino 1 = comum)	o pino comum da rede resistiva vai no pad quadrado
Display apagado	display de ânodo comum, ou EEPROM do display não gravada	use display **cátodo comum**; grave `DISPLAY.bin`
Display tremendo/fantasma	varredura lenta ou rápida demais	ajuste R10/C5 (padrão 100 kΩ/15 nF ≈ 470 Hz)
Comportamento aleatório em clock alto	ruído na alimentação	acrescente 100 µF/10 V perto do J1 de cada placa, use cabos curtos

8.4 Medidas de referência

- Alimentação em cada placa: **4,75 V a 5,25 V** (família 74LS).
- Nível alto TTL: ≥ 2,4 V; nível baixo: ≤ 0,4 V (com LEDs nas saídas, alto pode ficar entre 3 V e 4 V — normal).
- Corrente total típica: 1,0–1,4 A.

Lista de materiais completa - Computador 8 Bits v3.0

> Autor do projeto: **Marcos Silva dos Santos** (MarcosObel). Gerada automaticamente a partir de `tools/gerador-kicad/designs.py`.

Compre **10 % a mais** de resistores, capacitores, LEDs e barras de pinos, e **1 CI reserva** de cada tipo mais usado (74LS173, 74LS245, 74LS157).

Totais (todas as 13 placas)

Quantidade	Componente
2	CI 74LS00 (DIP-14)
4	CI 74LS04 (DIP-14)
2	CI 74LS08 (DIP-14)
1	CI 74LS132 (DIP-14)
1	CI 74LS138 (DIP-16)
1	CI 74LS139 (DIP-16)
4	CI 74LS153 (DIP-16)
4	CI 74LS157 (DIP-16)
3	CI 74LS161 (DIP-16)
9	CI 74LS173 (DIP-16)
6	CI 74LS245 (DIP-20)
2	CI 74LS283 (DIP-16)
4	CI 74LS32 (DIP-14)
4	CI 74LS86 (DIP-14)
2	CI 74S189 (DIP-16)
4	CI AT28C16 (DIP-24)
2	CI NE555 (DIP-8)
1	CI NE556 (DIP-14)
1	1N5822
106	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)
1	Borne 5 V
3	Botao tactil 6x6 mm (4 terminais)
57	Capacitor 100nF
4	Capacitor 10nF
1	Capacitor 10uF eletrolitico
1	Capacitor 10uF tantalo
1	Capacitor 15nF
1	Capacitor 1nF
1	Capacitor 330nF
1	Capacitor 470uF 10V eletrolitico
1	Chave DIP 4 vias
1	Chave DIP 8 vias
4	Chave deslizante SPDT 3 terminais passo 2,54 mm
1	Display 7 segmentos 4 digitos 0,4" catodo comum (12 pinos)
1	Jack DC 5,5x2,1 (5 V)
7	LED amarelo 3mm
1	LED azul 3mm

Quantidade	Componente
59	LED verde 3mm
10	LED vermelho 3mm
1	PTC 3 A (MF-RG300)
1	Potenciometro 100k linear 9 mm (Alps RK09K)
1	Rede resistiva SIP-5 1k comum no pino 1 (4 resistores)
1	Rede resistiva SIP-9 1k comum no pino 1 (8 resistores)
1	Resistor 10 1/4 W
8	Resistor 100 1/4 W
1	Resistor 100k 1/4 W
6	Resistor 10k 1/4 W
14	Resistor 1k 1/4 W
3	Resistor 330 1/4 W
1	Resistor 330k 1/4 W
65	Resistor 470 1/4 W

Soquetes (recomendados para todos os CIs)

Quantidade	Item
18	Soquete DIP-14 (300 mil)
26	Soquete DIP-16 (300 mil)
6	Soquete DIP-20 (300 mil)
4	Soquete DIP-24 largo (600 mil) - ou ZIF 24 pinos
2	Soquete DIP-8 (300 mil)

Total de circuitos integrados: **56** (+ 1 display).

Itens gerais

Quantidade	Item
1	Fonte 5 V regulada, 2 A ou mais, plugue P4 5,5 x 2,1 mm (centro positivo)
52	Cabo jumper femea-femea de 4 vias, 20 cm (ou "cabo flat 4 vias" com conectores Dupont)
1	Base de MDF/acrilico ~ 60 x 45 cm
60	Espacador M3 x 10 mm + parafuso + porca (4 por placa)
1	Gravador de EEPROM (TL866II Plus / XGecu T48) - para as 4 EEPROMs 28C16
-	Placa de fenolite/fibra de vidro cobreada dupla face (ver tamanhos em hardware/README.md)
-	Percloreto de ferro, papel couche/glossy, fio rigido 0,6 mm (vias), estanho, fluxo

Por placa

Clock (`CLOCK`)

Qtd	Componente	Referencias
1	Capacitor 10uF tantalo	C1
2	Capacitor 10nF	C2 C3
1	Capacitor 330nF	C4
3	Capacitor 100nF	C5 C6 C7
2	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2
1	LED azul 3mm	LED1
1	Resistor 1k 1/4 W	R1
1	Resistor 10 1/4 W	R2
1	Resistor 10k 1/4 W	R4
1	Resistor 330k 1/4 W	R5
1	Resistor 470 1/4 W	R6
1	Potenciometro 100k linear 9 mm (Alps RK09K)	RV1
1	Botao tactil 6x6 mm (4 terminais)	SW1
1	Chave deslizante SPDT 3 terminais passo 2,54 mm	SW2
1	CI NE556 (DIP-14)	U1
1	CI 74LS00 (DIP-14)	U2
1	CI 74LS04 (DIP-14)	U3

Reset (`RESET`)

Qtd	Componente	Referencias
1	Capacitor 10uF eletrolitico	C1
1	Capacitor 100nF	C2
2	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2
1	LED vermelho 3mm	LED1
2	Resistor 10k 1/4 W	R1 R2
1	Resistor 1k 1/4 W	R3
1	Botao tactil 6x6 mm (4 terminais)	SW1
1	CI 74LS132 (DIP-14)	U1

Contador de Programa (`PC`)

Qtd	Componente	Referencias
2	Capacitor 100nF	C1 C2
5	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5
4	LED verde 3mm	LED1 LED2 LED3 LED4

Qtd	Componente	Referencias
4	Resistor 470 1/4 W	R1 R2 R3 R4
1	CI 74LS161 (DIP-16)	U1
1	CI 74LS245 (DIP-20)	U2

RAM e MAR (`RAM`)

Qtd	Componente	Referencias
12	Capacitor 100nF	C1 C2 C3 C4 C5 C6 C7 C8 C9 C10 C11 C12
1	Capacitor 1nF	C13
1	Capacitor 10nF	C14
5	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5
9	LED verde 3mm	LED1 LED2 LED3 LED4 LED5 LED6 LED7 LED8 LED14
4	LED amarelo 3mm	LED9 LED10 LED11 LED12
1	LED vermelho 3mm	LED13
14	Resistor 470 1/4 W	R1 R2 R3 R4 R5 R6 R7 R8 R9 R10 R11 R12 R14 R15
3	Resistor 1k 1/4 W	R13 R16 R18
1	Resistor 10k 1/4 W	R17
1	Rede resistiva SIP-9 1k comum no pino 1 (8 resistores)	RN1
1	Rede resistiva SIP-5 1k comum no pino 1 (4 resistores)	RN2
1	Chave DIP 4 vias	SW1
1	Chave deslizante SPDT 3 terminais passo 2,54 mm	SW2
1	Chave DIP 8 vias	SW3
1	Botao tactil 6x6 mm (4 terminais)	SW4
2	CI 74S189 (DIP-16)	U1 U2
2	CI 74LS04 (DIP-14)	U3 U4
1	CI 74LS245 (DIP-20)	U5
1	CI 74LS173 (DIP-16)	U6
4	CI 74LS157 (DIP-16)	U7 U8 U9 U10
1	CI 74LS00 (DIP-14)	U11
1	CI NE555 (DIP-8)	U12

Reg. Instrucao (`IR`)

Qtd	Componente	Referencias
3	Capacitor 100nF	C1 C2 C3
5	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5
8	LED verde 3mm	LED1 LED2 LED3 LED4 LED5 LED6 LED7 LED8
8	Resistor 470 1/4 W	R1 R2 R3 R4 R5 R6 R7 R8

Qtd	Componente	Referencias
2	CI 74LS173 (DIP-16)	U1 U2
1	CI 74LS245 (DIP-20)	U3

Decodificador de Controle (`CD`)

Qtd	Componente	Referencias
3	Capacitor 100nF	C1 C2 C3
9	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5 J6 J7 J8 J9
2	Resistor 10k 1/4 W	R1 R2
3	CI AT28C16 (DIP-24)	U1 U2 U3

Contador de Passos (`FETCH`)

Qtd	Componente	Referencias
2	Capacitor 100nF	C1 C2
3	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3
3	LED amarelo 3mm	LED1 LED2 LED3
5	LED verde 3mm	LED4 LED5 LED6 LED7 LED8
3	Resistor 330 1/4 W	R1 R2 R3
5	Resistor 470 1/4 W	R4 R5 R6 R7 R8
1	CI 74LS161 (DIP-16)	U1
1	CI 74LS138 (DIP-16)	U2

Registrador A (`ACC`)

Qtd	Componente	Referencias
3	Capacitor 100nF	C1 C2 C3
6	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5 J6
8	LED verde 3mm	LED1 LED2 LED3 LED4 LED5 LED6 LED7 LED8
8	Resistor 470 1/4 W	R1 R2 R3 R4 R5 R6 R7 R8
2	CI 74LS173 (DIP-16)	U1 U2
1	CI 74LS245 (DIP-20)	U3

Registrador B (`B`)

Qtd	Componente	Referencias
3	Capacitor 100nF	C1 C2 C3
6	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5 J6
8	LED verde 3mm	LED1 LED2 LED3 LED4 LED5 LED6 LED7 LED8
8	Resistor 470 1/4 W	R1 R2 R3 R4 R5 R6 R7 R8
2	CI 74LS173 (DIP-16)	U1 U2

Qtd	Componente	Referencias
1	CI 74LS245 (DIP-20)	U3

ULA (`ALU`)

Qtd	Componente	Referencias
18	Capacitor 100nF	C1 C2 C3 C4 C5 C6 C7 C8 C9 C10 C11 C12 C13 C14 C15 C16 C17 C18
9	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5 J6 J7 J8 J9
8	LED verde 3mm	LED1 LED2 LED3 LED4 LED5 LED6 LED7 LED8
8	Resistor 470 1/4 W	R1 R2 R3 R4 R5 R6 R7 R8
2	CI 74LS283 (DIP-16)	U1 U2
4	CI 74LS86 (DIP-14)	U3 U4 U10 U12
2	CI 74LS08 (DIP-14)	U5 U6
4	CI 74LS32 (DIP-14)	U7 U8 U9 U11
4	CI 74LS153 (DIP-16)	U13 U14 U15 U16
1	CI 74LS245 (DIP-20)	U17
1	CI 74LS04 (DIP-14)	U18

Registrador de Saida (`OUT`)

Qtd	Componente	Referencias
2	Capacitor 100nF	C1 C2
6	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3 J4 J5 J6
8	LED verde 3mm	LED1 LED2 LED3 LED4 LED5 LED6 LED7 LED8
8	Resistor 470 1/4 W	R1 R2 R3 R4 R5 R6 R7 R8
2	CI 74LS173 (DIP-16)	U1 U2

Display Decimal (`DISPLAY`)

Qtd	Componente	Referencias
4	Capacitor 100nF	C1 C2 C3 C4
1	Capacitor 15nF	C5
1	Capacitor 10nF	C6
1	Display 7 segmentos 4 digitos 0,4" catodo comum (12 pinos)	DS1
3	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J1 J2 J3
1	Resistor 1k 1/4 W	R1
8	Resistor 100 1/4 W	R2 R3 R4 R5 R6 R7 R8 R9
1	Resistor 100k 1/4 W	R10
1	Chave deslizante SPDT 3 terminais passo 2,54 mm	SW1
1	CI AT28C16 (DIP-24)	U1

Qtd	Componente	Referencias
1	CI NE555 (DIP-8)	U2
1	CI 74LS161 (DIP-16)	U3
1	CI 74LS139 (DIP-16)	U4

Backplane (`BACKPLANE`)

Qtd	Componente	Referencias
1	Capacitor 470uF 10V eletrolitico	C1
1	Capacitor 100nF	C2
1	1N5822	D1
1	PTC 3 A (MF-RG300)	F1
1	Jack DC 5,5x2,1 (5 V)	J1
1	Borne 5 V	J2
45	Barra de pinos macho 1x4 2,54 mm (90 graus ou reta)	J3 J4 J5 J6 J7 J8 J9 J10 J11 J12 J13 J14 J15 J16 J17 J18 J19 J20 J21 J22 J23 J24 J25 J26 J27 J28 J29 J30 J31 J32 J33 J34 J35 J36 J37 J38 J39 J40 J41 J42 J43 J44 J45 J46 J47
1	LED verde 3mm	LED1
8	LED vermelho 3mm	LED2 LED3 LED4 LED5 LED6 LED7 LED8 LED9
1	Resistor 470 1/4 W	R1
8	Resistor 1k 1/4 W	R2 R3 R4 R5 R6 R7 R8 R9
1	Chave deslizante SPDT 3 terminais passo 2,54 mm	SW1